Informe Técnico: Asistente de Redacción con IA para Planes y Proyectos

Módulo 2 - Técnicas y Herramientas Modernas

Bressan, Cruz, Don, Fischer y Ludueña

25/09/2026

Contents

Introducción a la Inteligencia Artificial y Algoritmos Bioinspirados **1**

Evolución Arquitectónica: De la IA Clásica a los Transformers **2**
El problema de la secuencialidad . . . 2
El salto tecnológico: la arquitectura Transformer . . . 2

Fundamentos Matemáticos: Espacio Vectorial y Estocasticidad **2**
Proximidad Vectorial Semántica . . . 2
Condición de aplicación en la redacción . . . 3

Etapas del Procesamiento del Lenguaje **3**
Tokenización . . . 3
Embeddings . . . 3
Mecanismo de Autoatención (*Self-Attention*) . . . 3

El Parámetro de Temperatura (T) y Demostración en R **3**
Función de Softmax con parámetro de Temperatura . . . 4
Datos de entrada . . . 4
Comparación de temperaturas . . . 4
Caso 1: Temperatura baja ($T = 0.2$) . . . 4
Caso 2: Temperatura estándar ($T = 1.0$) . . . 4
Resultados . . . 4

Limitaciones, Fiabilidad y Sistemas Avanzados **5**
Errores y Alucinaciones . . . 5
Comportamiento Emergente . . . 5
Sistemas RAG e IA Agéntica . . . 5

Ingeniería de Prompts (Instrucciones) para Proyectos **6**

Generación de Imágenes mediante IA **6**

Introducción a la Inteligencia Artificial y Algoritmos Bioinspirados

En el ámbito de la ingeniería y la computación, es fundamental comprender de dónde surgen los algoritmos que utilizamos. Desarrollar algoritmos no implica necesariamente imitar el pensamiento humano de forma

exclusiva; existen algoritmos que se basan en el comportamiento de los hormigueros o en el de las abejas. Las redes neuronales, por su parte, intentan replicar el método de cómputo que se realiza en el cerebro humano.

Un ejemplo clásico de optimización estocástica son los algoritmos genéticos, los cuales son sumamente utilizados en la actualidad. La fundamentación de estos algoritmos radica en el concepto de la evolución biológica:

- Este algoritmo inicia su proceso con un conjunto de datos inicial denominado **población**.
- Dicha población está compuesta por individuos, los cuales terminan siendo vectores que pueden representar, por ejemplo, una imagen.
- El mecanismo permite cruzar vectores (simulando que tienen hijos entre sí) para iterar y formar una nueva generación con características optimizadas.

Evolución Arquitectónica: De la IA Clásica a los Transformers

Para comprender por qué las herramientas modernas de redacción funcionan con la fluidez actual, debemos analizar la evolución del procesamiento del lenguaje.

El problema de la secuencialidad

En la inteligencia artificial clásica, tareas complejas como las traducciones se realizaban mediante redes neuronales que buscaban patrones en los lenguajes. Sin embargo, estas redes operaban bajo una limitación fundamental:

- Nunca podían ser rápidas porque imitaban el procesamiento secuencial, leyendo palabra a palabra.
- El sistema no lograba asimilar el significado completo hasta no avanzar y procesar la totalidad de la frase.
- Esta barrera técnica impedía la viabilidad de las traducciones instantáneas que requerimos hoy en la práctica profesional.

El salto tecnológico: la arquitectura Transformer

La solución a este cuello de botella algorítmico surgió cuando Google desarrolló una red neuronal con una nueva arquitectura denominada **Transformer**. Este hito, publicado en 2017 bajo el artículo *"Attention is All You Need"*, es el origen de la revolución de la IA generativa actual.

La principal justificación de su eficiencia y velocidad radica en que el procesamiento se realiza en paralelo, a diferencia de los modelos secuenciales previos. La IA generativa moderna se basa enteramente en el uso de estos Transformers.

Fundamentos Matemáticos: Espacio Vectorial y Estocasticidad

Como ingenieros, no debemos tratar a la IA como una "caja negra" mágica, sino como un sistema matemático determinable en sus bases, aunque con un comportamiento emergente complejo.

En el fondo, la IA generativa opera mediante un proceso estocástico avanzado (donde el resultado es probabilístico) sobre un espacio vectorial de alta dimensión.

Cuando introducimos una instrucción (*prompt*), el sistema no razona como un humano, sino que traduce todo a matrices y vectores, ejecuta operaciones matemáticas y nos devuelve resultados probabilísticos.

Proximidad Vectorial Semántica

El modelo de lenguaje agrupa el conocimiento ubicando conceptos en el espacio vectorial.

- Conceptos relacionados (como control de acceso, identificación biométrica y huella digital) se representan con vectores cercanos en el espacio.

- Por el contrario, un vector no relacionado (como el pronóstico meteorológico) se ubica en una posición lejana.
- Debido a la inmensa cantidad de datos procesados, estas relaciones matemáticas generan una extensa "nube" de información.

Condición de aplicación en la redacción

Si al redactar un informe técnico iniciamos el *prompt* con frases como "por favor decime...", estamos introduciendo lenguaje propio de redes sociales que no aparece en textos técnicos rigurosos. Esto provoca que los vectores de búsqueda se alejen matemáticamente de las fuentes académicas o técnicas que necesitamos.

Es un error conceptual antropomorfizar a la herramienta; no es una persona a la que se le pide un favor, es un sistema ejecutando cálculos.

Etapas del Procesamiento del Lenguaje

Para que el modelo procese nuestro texto, se llevan a cabo tres fases secuenciales:

Tokenización

- Es una fase técnica de muestreo y cuantización de los datos.
- El texto (por ejemplo, "Mecatrónica pura") se trunca y se corta utilizando una metodología específica, convirtiéndolo en un número entero.
- Estos números enteros constituyen el lenguaje real que el modelo de lenguaje procesa.
- En términos de eficiencia computacional y costos: cuando se cobran los tokens, lo que se está cobrando es precisamente este proceso de muestreo y cuantización.
- Al adjuntar archivos, enviar un archivo Word (`.doc`) gasta muchos más tokens debido a los códigos ocultos de formato que el modelo también debe procesar.
- Este costo computacional se ahorra si se envía el archivo en formato `.pdf`, y el consumo de tokens es aún menor si se utiliza formato de texto plano como `.md`.

Embeddings

- Una vez que el texto se convierte a números enteros, se pasa a la fase de *embedding*, que conforma el espacio vectorial.
- En esta fase, todos los vectores apuntan a distintos tipos de conocimiento.
- La cercanía entre los vectores es la medida que representa el parecido en el significado de las palabras procesadas por el modelo.

Mecanismo de Autoatención (***Self-Attention***)

- La innovación clave del Transformer es el mecanismo de "atención" o "autoatención", que le otorga la capacidad de entender rápidamente la relación de todas las palabras entre sí.
- Este mecanismo se fundamenta en una ecuación matemática que calcula y establece probabilidades de relación entre las palabras en función del contexto.
- El sistema calcula cuál es la probabilidad de que una cierta frase termine de una manera determinada basándose en el contexto provisto.
- El hecho de que ciertas palabras sean estadísticamente más probables que otras puede hacer que el modelo parezca "descuidado", ya que prioriza la probabilidad sobre la veracidad factual.

El Parámetro de Temperatura (T) y Demostración en R

Dentro de la ecuación matemática subyacente que rige el Transformer, el parámetro T (temperatura) es fundamental porque determina hacia dónde está dirigida la búsqueda en el espacio vectorial. En términos

prácticos, altera la distribución de probabilidad (usualmente a través de una función *Softmax*).

A menor temperatura, el modelo se vuelve determinista y escoge estrictamente el vector más cercano; a mayor temperatura, aplana la probabilidad permitiendo opciones más alejadas (mayor "creatividad" o aleatoriedad).

A continuación, ejecutaremos un bloque de código en R para demostrar numéricamente cómo afecta el parámetro T a las probabilidades de salida de un modelo al calcular el siguiente token.

Función de Softmax con parámetro de Temperatura

```r
# Función de Softmax con parámetro de Temperatura (T)
# Explica matemáticamente cómo se calculan las probabilidades en un Transformer

softmax_con_temperatura <- function(logits, temperatura) {

  # Asegurar que T no sea cero para evitar indeterminación
  t_val <- ifelse(temperatura == 0, 1e-6, temperatura)

  # Aplicamos la temperatura a los logits escalándolos
  logits_escalados <- logits / t_val

  # Para estabilidad numérica en R, restamos el máximo
  exp_logits <- exp(logits_escalados - max(logits_escalados))
  probabilidades <- exp_logits / sum(exp_logits)

  return(probabilidades)
}
```

Datos de entrada

```r
# Supongamos que el modelo calculó las siguientes
# "energías" o afinidades (logits) para las próximas
# 3 posibles palabras en un informe técnico

logits_vocabulario <- c(5.0, 3.5, 1.0)
nombres_palabras <- c("óptimo", "adecuado", "irrelevante")
```

Comparación de temperaturas

Caso 1: Temperatura baja ($T = 0.2$)

```r
# Búsqueda altamente determinista
probs_baja <- softmax_con_temperatura(logits_vocabulario, 0.2)
```

Caso 2: Temperatura estándar ($T = 1.0$)

```r
# Búsqueda balanceada
probs_media <- softmax_con_temperatura(logits_vocabulario, 1.0)
```

Resultados

```r
resultados <- data.frame(
  Palabra = nombres_palabras,
```

```
  Logit = logits_vocabulario,
  Probabilidad_T_0.2 = round(probs_baja, 4),
  Probabilidad_T_1.0 = round(probs_media, 4)
)

knitr::kable(
  resultados,
  caption = "Probabilidades de salida para diferentes valores de temperatura"
)
```

Table 1: Probabilidades de salida para diferentes valores de temperatura

Palabra	Logit	Probabilidad_T_0.2	Probabilidad_T_1.0
óptimo	5.0	0.9994	0.8055
adecuado	3.5	0.0006	0.1797
irrelevante	1.0	0.0000	0.0148

Observación del experimento computacional: Como se observa en la ejecución, a menor temperatura, la probabilidad colapsa sobre la opción más predecible. Esto justifica por qué en aplicaciones de ingeniería exigimos bajas temperaturas.

Limitaciones, Fiabilidad y Sistemas Avanzados

Errores y Alucinaciones

Como ingenieros, debemos diseñar operando bajo condiciones de incertidumbre conocida. Al día de hoy, existe una probabilidad del 5% al 10% de que el resultado entregado por la IA sea erróneo.

- Estos errores pueden manifestarse como errores de tipeo, errores matemáticos o referencias bibliográficas mal escritas e inexistentes.
- La respuesta suele presentarse de forma convincente, pero eso no garantiza que sea técnicamente correcta.
- Es un requisito inexcusable verificar siempre la información y exigir al sistema: "Y dame la referencia de donde lo obtuviste".
- La inteligencia artificial no reemplaza bajo ninguna circunstancia el criterio profesional; la respuesta debe ser validada técnicamente antes de integrarse a cualquier decisión de diseño o proyecto.

Comportamiento Emergente

Los modelos Transformers son tan complejos que a menudo operan como una "caja negra" o máquina negra. Esto genera un comportamiento emergente en el cual no se sabe ni se puede predecir exactamente qué es lo que el modelo podría llegar a hacer o resolver.

Sistemas RAG e IA Agéntica

Para mitigar los errores estocásticos en el ámbito técnico, utilizamos arquitecturas adicionales:

- **RAG (Generación Aumentada por Recuperación):** Este método consiste en entregarle al modelo una base de conocimiento externa y específica. Al hacer esto, la IA generativa restringe su movimiento estrictamente a los datos que nosotros le proporcionamos, arrojando una respuesta final que depende directamente de esta información curada.

- **IA Agéntica:** Consiste en agentes que tienen la posibilidad de trabajar de manera autónoma. Estos agentes deben desplegarse en áreas que no representen peligro. En áreas críticas, se debe establecer una restricción explícita de supervisión, indicando: "hacé tal cosa, pero antes de llevarlo a cabo, pedime mi autorización", lo que permite al ingeniero revisar el trabajo y aprobarlo.

Ingeniería de Prompts (Instrucciones) para Proyectos

Dado que el sistema basa sus cálculos probabilísticos en lo que introducimos, el grado de especificidad de nuestro *prompt* es inversamente proporcional a la incertidumbre del resultado. Todo parámetro que no se especifique quedará librado al azar de los cálculos probabilísticos.

Para redactar planes y proyectos, un *prompt* debe contener como mínimo **6 parámetros**:

1. **Rol esperado:** Es vital para ubicar correctamente los vectores en el espacio de conocimiento adecuado. Ejemplos: "Actúa como un Ingeniero Industrial especializado en..." o "Actúa como un equipo en el que trabaja un psicólogo, un matemático...".
2. **Objetivo:** Se debe especificar de manera perfecta y clara qué es exactamente lo que se desea que el sistema haga.
3. **Datos disponibles:** Son los datos de entrada con los que operará la máquina. Si se cuenta con especificaciones, la claridad es fundamental; un exceso de texto confuso impedirá que el modelo sepa cómo proceder.
4. **Restricciones:** Establece los límites técnicos o condiciones de contorno que la solución generada debe respetar rigurosamente. Para trabajos muy específicos, se requiere que el usuario sea un experto en el área para poder indicarle "absolutamente TODO".
5. **Producto a obtener:** Definir qué entregable físico o digital se espera.
6. **Formato de salida:** Se aconseja utilizar ejemplos como patrón de referencia. Proporcionar un ejemplo de formato le es de gran utilidad al modelo y puede suministrarse tanto en el propio texto del *prompt* como adjuntando un documento.

Generación de Imágenes mediante IA

En el caso de diagramas o visualizaciones anexas al informe técnico, la generación de imágenes sigue una lógica distinta pero igualmente fundamentada en procesos matemáticos.

- La generación de imágenes funciona como una generación aleatoria. Irónicamente, mientras menos se especifique en la instrucción, la calidad visual final suele ser percibida como mejor.
- El modelo inicia el proceso partiendo de una imagen que se compone exclusivamente de ruido puro, donde no existe ninguna figura reconocible ni nada identificable.
- El algoritmo busca patrones en su base de datos de imágenes para identificar formas, colores, objetos, estilos, perspectivas e iluminación.
- Al ingresar el *prompt*, el sistema inicia un proceso de reducción iterativa de ruido (le saca ruido) apoyado en los parámetros indicados, hasta lograr identificar elementos reconocibles.
- Tras reducir el ruido, procede a la etapa de añadir detalles, constituyendo un proceso aleatorio fundamentado en el aprendizaje previo de la red.
- A partir de dichos patrones, la IA construye finalmente una imagen única que guarda coherencia con la instrucción dada.

Este proceso de identificación y generación de parámetros se estructura en base a:

- **Silueta**
- **Estructura**
- **Alta fidelidad**

Para maximizar el rigor en la creación de imágenes para un documento técnico, la estructura de un *prompt* visual efectivo debe incluir sistemáticamente:

1. **Objeto o Escena**
2. **Acción o Situación**
3. **Contexto o Entorno**
4. **Atributos Visuales**
5. **Estilo Visual**
6. **Composición**
7. **Iluminación y atmósfera**
8. **Nivel de detalle**